Video Transcripts

Module 14: Java and Debezium

Video 1: Introduction (03:55)

Java is something we haven't addressed yet. In many ways we have tried to avoid it. It is a language that imposes a lot of structure on you. It is a very different environment. It is mostly in legacy systems at this point and so we have not addressed it. However, when it comes to data platforms, especially those from the Apache Software Foundation, there's a great deal of Java there and there are many projects that are written in Java. Just to list a few, Spark, Kafka, Cassandra, Hadoop, Elasticsearch, HDFS, CouchDB and many others. And so because of it, it is good to have some basics about the language, some basics, and to how to set up an environment to run in. And so, that's what we will be doing in this section. This will be a mini crash course on Java to be able to get some of those basics that I mentioned that would at least help you to run some of those projects that require a Java platform.

When it comes to doing change data capture, as you can imagine, doing it by hand, it's not such a great idea. You can certainly do it, we did it, but it is more of an educational type of approach. We want to understand the issues. And although there are shops out there that maintain this type of systems by hand, there are better alternatives. It feels at times like a Rube Goldberg machine, where you have too many moving pieces and your risk of something breaking is too high. Now, as you can imagine, change data capture is a problem that is well recognized, and because of it there have been a number of projects that have dedicated efforts and come up with great solutions. Now, there's a number of these and we will sample these in this section. We'll start off with ones that are in Python and that are a great fit to where we have focused our efforts. They're light, they're easy, you bring them in, and you can monitor a system pretty well.

Now, after we do that, we will be looking at some of the really industrial solutions, meaning these scale at the highest level and they are written in platforms such as Java. Here, we will use containers once again to be able to forgo all of the installation that we would need to be able to get that going. But it's a great way to observe a full change data capture solution, that is, the one that is used at the highest level, meaning at the highest scale when it comes to data. So, now let's

go ahead and jump in.

As we have seen, many of the data platforms are from the Apache Foundation. And if you were to take a closer look at the many, many projects that the Apache Foundation has, you'll notice that a lot of them are written in Java. And as you can see here, some of the major building blocks when it comes to data engineering projects like Spark, Kafka, Cassandra, they're all written in Java. Now, there's something to say about Java in that although it is younger than Python, you can see there the dates, it has a lot more structure. And because of it, it is not one of the most loved languages.

As you can see there, it is certainly not in the top half. Now, more than that, as you can see as well, it's also in the top half of the most dreaded languages. However, even though it has those shortcomings, it is widely used in the market. You can see there that it's at the top of popularity, meaning the projects and the number of developers that exist out in the market. And so because of it, we're going to take a look at some of the basics as you're bound to touch Java projects sooner or later when it comes to your data engineering career. And this is good knowledge to have.

Video 2: Java Containers (04:02)

Setting up a Java development environment can be quite a pain. Now luckily for us, we can use a container to do precisely that. And as you can see here, we're going to be doing the following. We'll start off by creating an interactive container for Java development. As you can see there, we're going to be using the image called 'openjdk', and we're going to be using version 11, and we're going to open up that container into the bash. Now inside of it, we're going to run a couple of commands to update the packages.

As you can see there, the very first line and then the second one to install an editor. This is something that you'll frequently want to do when you have a very trim container that doesn't even have an editor so that you can do a little bit of testing. In this case, we're simply going to set-up this to illustrate how to how to set up our development environment, and later we will write code into it. So, let's go ahead and paste that instruction into the command line. Let's go ahead and run it. And my machine right now is a clean machine. I don't have any containers, so it's going to go ahead and download them all.

And as you can see there, that was pretty fast. Also note that we are now at the command

prompt of the container. So, you can see up here, this is my machine, this is the host, this is where I ran the instruction. And over here, I am setting as root inside of the container. So, the very first thing that I'm going to do is not that, but I'm going to go ahead and paste that instruction. This is simply going to update the packages.

And it looks like I included there the previous line, but you can see there now that the packages are updating. Now, I'm going to go ahead and add the editor. Now as you can see there that was installed. If we list our files, you can see there that we are at the root of the file system. I'm going to go ahead and move into home, and as you can see there, home is empty. I'm simply going to demonstrate here creating a file with our editor. This is a commonly found editor in most Linux distributions. So, I'm going to go ahead and create here a 'hello.txt'.

And as you can see there, I am now inside of the editor. All I will say is 'Hello World', in here. Then I'm going to go ahead and save my file. And you can see there that the commands were at the bottom of the file. I'm going to go ahead and save it, show you that that file exists and that those are the contents. And if we were to go back to the file, you can see there the menu at the bottom of the screen that would allow you to perform some of the most frequently used commands.

I'm going to go ahead and exit at this point. I'm now going to go ahead and show you the container and as you can see here, I have opened up the dashboard on Docker and you can see there that my Java container is up and running. I can have the usual commands to stop it or restart it and that now it is prepared to do Java development. So, simply to overview, we created a container. The container was created with the given command. The name that we gave it was 'javahi'. That's why you saw that in the dashboard. And the rest of it is simply some of the options that we created the container with. And in this case, that last one is to log in or to jump in at the batch once we get that container up and running.

Video 3: Hello World! in Java (02:54)

Let's write a Java program, a 'Hello World!' program. As you can see here, I have just a handful of lines, and I want to illustrate some basic points. The very first one is that you bring additional functionality into the language by using the keyword import. And you can see there that I'm bringing in java.io, which is the input output package, the basic one from the language. Note there that I have also a public class, hello, which is defining my program and that the file name

that I use for this program has to match that class name. That is just a convention of the language. I also have there a main method, a hook into this program. And you can see there that I'm simply using System.out to be able to print the line, and that line is going to be Hello World. So, with that in mind, let's go ahead and write this program. So, here we are at the console inside of the container. I'm going to go ahead and clear the screen. I'm now going to go ahead and list my files. And there's a file we don't need there anymore, so I'm going to go ahead and delete it. Now, if I list my files, I will not have any of them as you can see there. And now I'm going to go ahead and use the nano editor to be able to create this file.

And as we need to match the class within the file, we're going to use the same name. So, this is hello.java, and now we're inside of the editor, I'm going to go ahead and paste my code. As you can see there, the editor gives me some nice syntax highlighting, as you can see. And we have there the parts of the program that we just overviewed. So, now I'm going to go ahead and exit, save my file, confirm that in fact has been saved. And as you can see there, it has. And now because Java is a language that needs to be compiled, it is not interpreted like say, JavaScript or Python. We need to compile this program. We do that by entering 'javac' and then the name of the file. We'll go ahead and hit return. And as you can see there that has been compiled. If we clear the screen again, list our files, you can see there that we have now a Hello.class and we can run that by entering 'java' and then the name Hello. And as you can see there, as expected, we get Hello World back. So, to overview, we have a file that we created that matches the class name within the file, we then wrote our code, we then compiled that code and we executed that code as we just saw.

Video 4: Data Types in Java (02:18)

Let's write a program that takes a look at some of the basic data types. As you can see there, we have Booleans, as in most languages, the true or false type of values, we have bytes, characters, integers, and a variety of those as well, some long, some short, and we have floats, and different types of floats as well, more or less precision. So, as you can see here, we have a simple program that is simply going to be a loop that is going through a number of iterations, and then we're writing to the console. And so, we are concatenating strings, we are using integers, and we're also using floats. So, let's take a look and see how we can write this program now within our terminal. Okay, so let's start off by writing our file. This is going to be Fahrenheit by writing. And inside of it, let's go ahead and paste our code. And as you can see there, we have just what we

had on the slides, I'm going to go ahead and remove the spaces. This is a kind editor that lets you know where you have spaces, where you might not have thought so. And the rest of it is just what we took a look at.

So, let's go ahead and save. Let's go ahead and compile. And once that has compiled, we can go ahead and run our program. And as you can see there, we get the temperature conversions between Fahrenheit and Celsius. Now to show you some of the inner workings of how types work, if we were to remove the decimal point there, save the file, compiled again, and here's where you get the difference between interpreted languages and compiles languages that every time you make a change you need to be able to recompile. And we were to run it once again, you'd see that the results you get there are all 0s, and that has to do with how those types are treated. I'll leave it up to you to dig in and find out more about them if you want to get deeper.

Video 5: Classes and Objects in Java (05:06)

If you're not familiar with classes and objects, and that wouldn't be surprising today, in fact, a lot of languages like JavaScript and Python take you a really long way without using them. And in fact many development platforms have removed them due to the additional friction and not really a lot of help that they can provide. But as you can imagine, because of what I've mentioned when it comes to the language of Java, that it would be deep at the heart of it and in fact it is. And so because of it, let's look at some of the basics and use here a couple of classes and bring them together.

So, when it comes to the overall template, you can see here that we could define a class as a thing. And there used to be in the old days deep hierarchies of these that were constructed. But keeping it simple, you can imagine here a classical thing and inside of it, it would have two main constructs, one of them being the public part, that is the methods, as you can see there, the functions that can define what operations can be done on this type of object and then some private ones that would be limited to be visible within the class.

And so in general you could construct the thing. Once you define this thing, you would be able to create instances of it or occurrences of it, create new ones, and then within it you would have some functions, some methods that would be defined and that would provide support to be able to carry out the classes work. So, if we took a look at a couple of these and created an example, you can see here that I have created a Person program and it's a classical person and

inside of it, I have a constructor which is a method inside that we use to initialize the class. And in this case, we want to be able to pass in a parameter to set the name of a person.

And then what we have what follows next, there is a get and a set method that would allow us to perform those operations, that is get a name or set a name. And separately from that, you have a consumer file that is going to consume that class. So, as you can see there, within the method inside of consumer, we have there a new instance being created of Person. And once we have that new instance, then we can invoke some of the methods inside of it, the ones that are public. And you can see there how I am invoking the name.

So, let's go ahead and write this up and we'll start off by creating the file. And in this case, the file is going to be Person.java. Inside of it, I'm going to go ahead and paste our code, and as I scroll up, you can see there the the class being defined. You can see there our private variable, our constructor, our method that we used to define or to instantiate the class and you can see the other two supporting methods. So, I'm going to go ahead and exit from here, save the file. Then I'm going to go ahead and compile it just to make sure that, that does in fact compile.

And as you can see there, that has been created. Now I'm going to create the Consumer class, and my file is going to be Consumer.java. And now I'm going to go ahead and paste the code and I'm going to go ahead and remove some of the unnecessary spacing there. And as you can see there, it simply instantiates a class Person, and then we call one of its methods. So, I'm going to go ahead and save, and then I'm going to go ahead and compile. And as you can see there that has now compiled. Now let's go ahead and run the Consumer program that consumes the Person class. And now I'm going to go ahead and hit 'return', and as you can see there, that instantiated the class. Let's set the name as Peter Parker, and then we were able to recall it. Now if we took a look at the file system, you can see there that I have both of the files we created and we also have the classes that were generated when we compiled each of those files. So, now it's your turn. Make sure you can replicate what I have done, make sure that you can create some new ones.

Just think of some scenarios, think of something that you want to have multiple of, create your own program. And one last note here, note that all of these files are inside of the same directory. So, in order for the path resolution in this case, for the class to be discovered, it needs to be given the way that I have written it. It needs to be in the same directory.

Video 6: Packages in Java (04:40)

One of the issues you're going to run into is that of complexity as your code grows. Now a very effective technique to be able to address this is to be able to break things apart to minimize the complexity, the amount of code that you have in any one file. Now we did this previously, however, we leveraged the implicit resolution that exists within the framework, within the language to be able to do that and we kept all the files in one directory. Now that doesn't scale very well. And what you want to do is you want to leverage some of what has been built in into the language. And as you can see here, you can do packages and then those packages can be imported as I'm illustrating here, within these two files.

You can see here that the person file at the top is defining a package called university. Now on the right-hand side you can see there that file is then importing that package in that class as you see there. Now what this means from a practical standpoint when it comes to the file system is that the university package is going to live inside of a directory called University. As you can see there above that panel that defines the code and that the consumer class is on a directory above. So, I'm explicitly saying this within that node at the right-hand side at the bottom that says person must be in subdirectory named University.

So, let's go ahead and implement this as I think it will be clearer than simply describing it. So, as you can see here, I have those files already within the directory. I'm going to go ahead and take a look at Person. Person has the code we expect it to have. If I look at Consumer, the same thing is true. You can see there that as well. Now we need to add that package definition and we need to create those directories. So, I'm going to go ahead and start here by creating that directory. So, I'm going to call a directory. I'm going to create a directory called University.

Then I'm going to go ahead and move the Person file into it, into University. Now if we look at it, it has been moved into it, and now we're going to go ahead and move into University. We're going to go ahead and confirm that, that file is there, and now I'm going to go ahead and open up that directory. Now the thing that we want to do here is we want to define that package, and the package is going to be called University, which is the name of the directory that it's in. I'm going to go ahead and then save that file, then confirm that, that change has been added. And you can see there that it has. I'm now going to go ahead and compile it. So, I'm going to go ahead and enter Javac, then Person. And as you can see there that has been compiled. I'm now going to move a directory back up. And so I'm back at consumer and you can see there the directory

called University. You can also list it this way so that it looks a little bit easier to read.

And now we're going to have to add those changes to Consumer. So, let's go ahead and open up that file. And here we're going to import. We're going to use the directory name University.Person, and then the rest of it is the same. All we're doing is adding visibility and bringing in that package. So, I'm going to go ahead and close, save, I'm going to go ahead and compile, in this case, Consumer. As you can see there, we have it already. So, let's go ahead and run it to confirm that what we did actually worked. So, let's go ahead. And as you can see there, we get the same result as before. That class is instantiated. That means we can import that package and then we can execute functionality within that new instance of the class. So, now it's your turn. Make sure you understand how packaging works within the Java framework and that you can repeat what I have done.

Video 7: Uploading and Downloading Files to and from Containers (04:35)

As we have seen, working in a container is a great strategy. It allows us to create a clean development environment on demand, gives us great speed, and we can bypass all of the painful configurations that need to be done by hand. However, at times you might want to interact with that container from your host. That is, you might want to copy some files into it or some files out of it. And so I've provided you here with the following that you can use to be able to do that. And let's go ahead and demonstrate it just to make sure that we are seeing it for what it is.

As you can see here, I have 3 windows. On the left-hand side I have a container, a Java development environment container, and on the right-hand side I have two windows, both of them on my local host. This is a sample directory in my desktop and below it I am in the same location but just at the terminal. I'm going to start off here by creating a file and I'm going to call that file hello.txt, and inside of it I'm simply going to say 'Hello World'. Then I'm going to go ahead and save that file, taking a look at the contents to make sure it is what we think it is.

Now as you can see there on the window above is just simply the finder, the graphical view of our file system. And you can see there that I have that file hello.txt. Now let's start out by copying that file to the container. And before I do so, let me go ahead and show you the files that I have here. I have three files, Consumer.class, Consumer.Java and a directory called University. That's all there is in that directory. Now I want to modify this command, that instruction that I have here to be able to push that new file that we just created called 'hello' into that directory.

So, I'm going to go ahead and use the name of my container. And as you can see here, I have just opened up the dashboard and you can see here that the name of our Java development environment container is called 'javahi'. So, I'm going to go ahead and use that here. So, I'm going to go ahead and modify my container to 'javahi' and then the name of the file that I'm going to be pushing in is called 'hello' and the path that I want to push it into is the home path. So, if we take a look at our path there, it is '/home'.

So, I'm going to go ahead and enter that now and let's go ahead and list our files here. And you can see here that we now have that file. And if I take a look at the contents, we have 'Hello world!' So, going back to the slides, as you can see there, I have copy one file to container. I have copying from container. So, let's do the opposite, copying from container. And in this case, I have already modified that command to match what we have here.

So, I'm going to go ahead and copy that file. So, I'm going to copy the file that I have here, which is the Consumer.java file. So, I'm going to go ahead and show you the contents of that. And now I'm going to go ahead and copy it locally. And as you can see there on top, that file just appeared. If I look at the contents of it, we would see the same thing that we did within the container. So, as you can see, it's pretty easy to be able to move back and forth into the container, out of the container. I did it for a single file, but you can see there below how to do it for the rest of the directory, all of its contents.

And so if you want to save your work as you go through this and not have it be deleted when you erase your container, this is the way to do it. And you might also want to do it, by the way, whenever you create something outside or you download it from GitHub and you just want to be able to push it in without having to discover how you do it within the container. So, now it's your turn. Go ahead and give it a try. This is a useful thing to know.

Video 8: Spring Boot: A Web Developing Application for Java (09:54)

Programming languages are pretty compact, so when it comes to creating an application, there's more that's needed. And because of it, communities create application frameworks and the Java domain. one of the most popular, one of the most widely used is Spring Boot. As you can see here, Spring is the application framework, and Spring Boot goes even farther. In fact, there's a website that they have created that makes it very easy for you to create an application. And so that's what we're going to do. We're going to start off by navigating to that website, creating that

application, downloading those files onto our host machine.

We will then create a container, that container will have everything that you need to run these applications. We will add to it an editor and also expose a port, as you can see. We're then going to upload that code that we have downloaded onto our host machine, and then we're going to make some configurations to do our 'Hello World', and then we're going to go ahead and run it. So, let's get started by navigating to that website and creating our application. And as you can see here on my desktop, I have opened up a browser. Next to it I have a directory file system. This is pointing to a folder on my desktop. You can see desktop sample on the path and then below it I have the same, except that I am at a terminal. And so, you can see they are the same path as well.

So, I'm going to go ahead and navigate to that address. And you can see here that I have start.spring.io, that is the address that I'm navigating to. And we're going to go ahead and select or leave the default, as you can see there, it is using the project Maven. It is using the language of Java, the version of Spring Boot that we're using, the project metadata. We're going to leave all of that as default, and we're simply going to add one dependency to be able to create a web server, and we're going to create within it our 'Hello world'. So, that's all we're going to do. We're then going to go ahead and generate this application. And as you can see, that file, that zip has been downloaded onto my machine, the host machine. Now, I'm going to go ahead and drag and drop that zip onto my sample directory, as you can see there. And so I have them here at my command line. I can list the files as well.

And so having done so, we're going to go ahead and move on to our next step, which is to create the container. Remember, we're going to expose a port and add an editor. And as you can see, here is the command line to be able to do that. I'm going to go ahead and run that. I'm going to go ahead and that's it. That's going to open up to a prompt. And you can see there that I'm using Maven, which is required for this environment. This project is from the Apache Software Foundation if you want to learn more about it. You can see there that I am exposing the port 8080 and that I'm naming that container javamaven.

Now, below it are just the lines to be able to add in editor once I create the container. I've opened my window full size and now I'm going to go ahead and paste that command onto it. And as you can see there, that moves pretty quickly through the installation and now we have our command prompt. If I list my files, you can see there that I am at the root, I have our command

prompt that I've just created. I'm now going to go ahead and install the editor.

And now that, that is installed, we have a container configured with the right development environment and an editor for us to use. So, our next step is to upload the code, and we will do so from the host. As you can see here, once again, I am at the desktop inside of the sample folder and we're going to be copying the files that are inside of demo and we're going to be copying those files to the home directory. I've chosen that one because it's empty. And so, now I'm going to go ahead and enter the command. And as you can see there that is Docker copy demo pointing to the files inside of the demo directory here locally on the host machine. And then I'm going to copy that to the container. The container is called javamaven and inside of it to the home directory.

So, once I hit that and enter it, then if we list our files here inside of the container, we can see the same files that we have on the host. Once we have moved our code into the container, then the next step that we want to do is that we want to modify slightly that application that we just uploaded. And we're going to do so by adding this code. This is a template application, as you might imagine, the one that we just downloaded. It's just a trimmed template to get you going on web applications, and all we're going to do is that we're going to replace the code that we will see in a file in a moment with the following.

This one is simply doing the 'Hello World', and as you can see there at the bottom, it takes two parameters and if no parameter is given, it simply returns 'Hello World'. Otherwise, it returns Hello and the name that was provided. So, let's go ahead and move to the container and inside of it we're going to navigate to the source. We start off by the source directory, the path that we're going to navigate through is painful and this has to do with how this platform does directories, how Java does name spaces. And as you can see there, I am going deep, deep into this code. I'm still going through it. Let's see, have I gotten to it? Yes. You can see there that I am at the demo application. And now I'm going to run my editor and edit that code. And as you can see there, I have the default code and we're going to replace that with what we saw on the slide. So, I'm going to go ahead and remove this code and replace it as so. Now, I'm going to go ahead and save this file, confirm my changes. And as we can see there, we have the code with that method there at the end that takes those parameters.

So, our last step now is to try it. As you can see there, we're going to access that from our host machine. We'll be opening up a browser that will hit that container in the port that it opened, the

8080 port that was opened at the very beginning and that is the same port that is running inside of the container within that application. But before we do that, we need to run the application and you can see here what the command is. Let's paste the command. And as that installs, I'm going to go ahead and open up a browser.

So, it looks like I have an error. Yes. And the reason for that is that I am not at the root of the application. So, let's go ahead and move into Home. It's looking for the dependencies, It's looking for the file that defines the application. And you can see there that pom is the one that does that, If we were to look through it, and I will leave that out to you, you could see the definition of the application. So, because it couldn't find it, it got stuck. So, I'm going to go ahead and run it now and the application is now up and running and listening.

I'm going to go ahead and navigate to the path. So, that would be localhost and we're going to go ahead and enter 'Hello'. And once we do, once we do, you can see there that we get 'Hello World!' There were two options or two parameters provided there, as you saw in that method inside of the application. If you provided no parameter accompanying just the path that we defined, then you got the default 'Hello World'. However, if we passed in a value, and in this case the value that we're going to pass in is going to be Peter, then we should get 'Peter'. And yes, we do.

Conceptually, we did quite a bit. So, let's review. We started out by using some scaffolding, the one that is provided for us by that website, start.spring.io, and we created there our application. We downloaded that onto our host machine. Then we created a container. We copied that code into the container. Then we added an editor, we edited the code, we created our 'Hello World', and then we ran that application in the container and accessed that externally. Now,, there's a lot going on here from an architecture standpoint. There are several packages being used, there are ports being mapped, there's a container being used on top of a host machine. So, go through this application in detail, make sure you understand all of those abstractions and what the architecture is. Definitely make that code run for you. Make some methods, make some changes, and make sure that what you think is taking place is in fact happening. So, now it's your turn.

Video 9: Event-Driven Approach for CDC (10:01)

Change happens. Tracking it, reacting to it is one of those things that is part of life. Change is inescapable. So, when it comes to systems, it's no surprise that the same thing is true. Change

data capture is a process that captures the changes made in data stores and ensures that those that care about those changes are notified. Now we created one of these by hands and we propagated that change through a number of data stores. This was the approach that we took. We made periodic queries and out of the ways that you could do it, do it yourself, DIY.

It's not a bad approach. This is architecturally the model that we had. We queried and we notified those that were interested, but I'm sure at some point you felt there were too many moving parts, and this was really a bit overwhelming. So, as it turns out, for performance reasons and others, an event-driven approach where you're reacting directly to the change as opposed to exploring for it, is much better. And as it turns out as well, most data stores keep a transaction log anyway. So, you can see there it is used to recover from failure, it is used to track changes. And the advantages are many.

One is that, it captures all changes, two is that it has low overhead as part of the operations of the data store. And a really, really great one is that it does not require database or application modifications. You don't have to touch your database, you don't have to touch the application that you're writing. And that is a great, great thing to have. Now the disadvantage is that you might have to write one of those for every type of database vendor there is. And that would be equally challenging because there are some other things that from having written one, you might recognize.

You might care about the message order that's taking place. You don't want to have robust publication and subscriptions. You want to have reliable and resilient delivery message transformations potentially. But overall, if you could do it, this would be a much better way. You have something that is tracking and flagging those events and then notifying those that care about it. Now fortunately, many of these have been written in our open source projects that you can readily use. We'll start off by looking at one in Python, but there are several others here that we will come back to and discuss.

So, let's start off here with the one on Python, as you can see, the architecture of the application that we're going to be writing is the following. We'll start off by creating a container, and that container will have MySQL database server. Then we will connect using the workbench and add to that database, a database, a table, and some data. And then we're going to be writing a Python program that is going to be doing an implementation of this package that does CDC for us. So, let's go ahead and get started by creating the container.

The container command that we're going to use is going to be the following as you can see here. We are going to create or we're going to use the port 3306, which is the default port for MySQL, and we're going to expose it to the host, the machine that we're running in. We're going to call it some SQL. We will set the password of MyNewPass. We're going to run that detached and we're going to base it on the MySQL image. And as you can see there, that was readily done, pretty speedy since I already had the image on my machine and now let's confirm in the dashboard.

Here is the container, as you can see and note that it's up and running, it's nice and green, and that it's running on port 3306. Next, let's open up a connection to our database using the workbench. I'm going to go ahead and enter the password that I just created and here we are now at the console ready for a new script to be created. I'm going to go ahead and paste some code. As you can see here, the very first block creates a database, on line five, we are using that new database and then we are creating a table called students with three fields, email, name, and city. So, let's go ahead and run that code. And as you can see there that was successful.

Let's go ahead and reload, and you can see there that we now have an education database and a table called Students. Next, let's open up Visual Studio and let's write a program to run against the database. Now before we write our program, let's go ahead and install the drivers, install the packages that are needed. I'm going to go ahead and open up the console. The first one that I'm going to install is PyMySQL. Note that I am specifying the version.

The second one is MySQL-replication, and note that those have been satisfied on my local environment. Now let's go ahead and write the code. We're going to start off here by bringing in the Binary log StreamReader. We're then going to set our settings that we're going to use for the connection, the host, this local, the port is 3306, the user is root, and then the password is the one we set, which is MyNewPass. We're then going to write a function and note that this is the one that creates that connection and makes calls the Binary log StreamReader.

Note that, we're passing in those settings that we just defined. Then we are going to give the server an ID and then we're setting the option of blocking to true. And you can see there on line 12 that if you do that, you can wait for the next event at the end of the stream. So, in other words, it hangs and listens. And then if we get any events, we're going to go ahead and dump them to the console. So, let's go ahead and finish this up here, with a call to that function. Let's go ahead and save.

Let's go ahead and open up the console and we're going to go ahead and run here. And the

program name is app.py. And as you can see there, we immediately got all of the changes that were recorded on that database from the time of creation. You can see there the events, the query events, the drop database if it exists, the create database if it doesn't exist, and finally the creation of the table, students. So, pretty fine detail of everything that is taking place. The great thing here is that we didn't have to write this ourselves.

This was written by the community and it's a package that we can leverage. Now since we're listening for the change, let's go ahead and add something to the database and see that reaction within this console. I'm going to go ahead here and make my windows smaller so I can have them side by side. So, now I have side by side, the workbench, and then I have Visual Studio and I have the console. I'm going to go ahead and make this window a little bit more roomy here by removing that side panel. And I'm going to go ahead and add some data to the student's table.

Now you can see here that I'm doing an insert into students and the values are going to be Peter and Peter Parker and then Cambridge, Massachusetts. So, I'm going to go ahead and add that. And if everything goes well, we should see the change on the right-hand side. I'm going to go ahead and do that now. And as you can see there, that change was captured. You can see there the details of the write rows event. You can see there the values of email, name, and city. And you can see there as well that if we had other parts of our program that cared about these changes, we can now notify them.

So, the great advantage here once again, is that we're not touching the application that we could be writing. We're not touching the database, we're not modifying it. We're simply bringing in a package that can do CDC by looking at the binary log record, something that the database is already doing anyway. So, this could be a great layer that we could have, that we could add that would propagate changes to any components that we had, any services that we had that would care about those changes. Now as you might imagine, this is a rich area of development. This is a rich area of research. And there are a lot of really interesting design trade-offs that can be discussed.

And if you want to look at a great discussion on these topics, you can take a look at DBLog made by Netflix or proposed by Netflix. There's even some technical applications that you can read if you're interested. But if you want to get a synopsis and an overview of the discussion, you can navigate to the links that I have provided here for you and you can learn more about it. So, now it's your turn. We went through several steps here. We created some containers, we populated a

database. We then hooked onto that database to the binary record and we were able to make updates and capture those in code that we have running on Python. So, now it's your turn. Make sure you can go through all of those steps before you go forward.

Video 10: Introduction to Debezium (01:13)

We previously wrote a change data capture application using Python. We used the package MySQL replication. Well, as it turns out, a lot of the work that is being done on data platforms is written in Java. And because of it, no surprise, the biggest packages for change data capture are written in Java as well. As you can see here, I have listed three of them, with the last one being one that has been proposed by Netflix. However, by far, the most widely used, the one that has been deployed the most, the one that has been built into some of the biggest applications in the market today, it's the Debezium, the open source project of Debezium.

So, we're going to be creating an application, a Spring Boot application that is going to implement this package. We're going to break that application into three steps. The very first one is going to be one where we create a network. The second one is one where we create the database. And the third one is going to be the Spring Boot application that uses Debezium.

Video 11: Network Setup for Debezium (01:32)

If we were to create two independent containers, and expose those ports to the host, we would be able to interact with each of them from the host. However, from inside of each of those containers, they would not be able to see each other. Now, this is exactly what we want to do when it comes to the application that we're building. So, we need to do something extra. And as you might imagine, the platform has recognized this, the Docker platform, and has introduce the concept of a network, a network within which you can put the containers so that not only can the host access those ports, but the containers can see each other as well. Now, creating a network is pretty straightforward.

As you can see here, you simply use the command 'docker network create' and then the name of the network that you want to use. In this case, we're going to be using 'netabel' for network able. So, here I am on the desktop, I have a folder within my desktop that is called sample, and I have a couple of windows opened up to the same place. So, I'm going to start off here on the bigger window and I'm going to enter that command. And as you can see here in a moment, that network will be created and is now available for us to use in the rest of this application for all of

the containers that we would want to add to it.

Video 12: Database Setup for Debezium (03:34)

Next up, is the database. We're going to go through three steps. The very first one is to create a Docker file. The second one is to create a Docker image and then from that image we're going to create a container. The Docker file itself is going to reference in SQL script file. The file is the following.

As you can see there is going to create a database, then a table within that database and then it's going to insert a row with the values that you see here. Now, that script is going to be referenced from the following Docker file. As you can see there, there is a line that says ADD customer.sql, and that SQL script is the one we just saw. Now, from there we're going to create an image. And as you can see there, we're going to call the image mysqlabel. So, now let's go ahead and create that, the command line. So, as you can see here on the upper right-hand corner, I have 3 folders. The very first one is the network setup. We have gone through that step. The second one is the database setup. And the third one is the Debezium, which we will do after this, after we set up the database. So, I'm going to go ahead and move into that folder. And as you can see here, we have the Docker file and we also have that customer.sql.

I have an additional file there that has some instructions as well. So, I'm going to go ahead and take a look at the Docker file simply to remind ourselves what's in it. And as you can see there, it is the same thing that we had on the slides and that it's referencing that customer.sql. If we take a look at that one, you can see there that that is the SQL script. So, I'm going to go ahead and clear the screen and then I'm going to go ahead and paste that command to create the Docker image. So, let's go ahead and run that. And as you can see there, that image has been created. If we were to list our images, we'd see there that the image we created is the latest. Now, that we have created our image comes the last step of the three, which is to create a container. Now, you can see there the command, and I have put the arguments in two different lines.

The very first argument after the first line is the name of this container and I'm going to call it mysqlserver. The port is going to be the default port. Then I'm going to put this container into the network that we created. That's an important point so that all of the containers that we add to that network can see each other. And you can see there that I'm referencing the name that we use, which is netabel for networkable. We're going to run that detach, that's -d. And then, we are

going to use the image that we just created, which is mysqlabel. So, let's go ahead and clear our screen, enter that command, hit return. And as you can see there that is going through and if we open our dashboard we can see there that MySQL server, based on mysqlabel, running on port 3306 is up and running.

Video 13: Using Debezium with Spring Boot (06:39)

Next up, we're going to be creating the app that is going to be doing the change data capture. This is a very simple app, a Spring Boot app, your Hello World app, that is simply going to have one more block of code to be able to listen to changes in the database using the Debezium. As before, we're going to be going through three stages. The very first one is to create a Docker file. Then from it, we're going to create a Docker image and then finally we're going to create a container based on that image. The Docker file itself is going to be referencing that code, that Spring Boot code that I mentioned, and you can download that code from the following repository within GitHub.

You can see here the Docker file, a very simple basic Docker file that references Maiven, which is needed to be able to develop and run Spring Boot applications. Aside from that, you can see there that it's going to copy that application into the container into a directory called 'tmp'. Now here at the bottom, we have the command to be able to create the image and note the name that I'm giving the image. The image is going to be called the debeziumabel. So, let's go ahead and create that image.

But before I do so, let me go ahead and move to the third folder. That third folder has the files that are needed to create this application, but I'm going to start off here by creating by creating the image. And so as you can see there, that was pretty fast and that has now been created. Now once we have our image, then we're going to create our container. And let's take a look here at some of the arguments. You can see there that I'm going to call that container, I'm going to call it the debeziumserver. I'm then going to add this container to the same network that we have been using, network netabel, which stands for network abel.

And then I'm going to be basing this container on the debeziumabel image that we just created. And we're going to be opening that container to the bash. So, let's go ahead and clear our terminal here, and I'm going to go ahead and paste and run that command. And in a moment, we should open up the prompt. If I hit 'return', if I list my files, you can see there that I have help, I

have pom, I have the source directory. And if I move within my host here into that third folder and list my files, you can see there that I have the same.

Well, actually let me move into app because that has the same. And so now that we have created our container and we have moved our code into it, the next step is going to be to run the application. But right before we do so, let me go ahead, and show you that additional block of the Debezium code that was added. And so I'm going to show you that folder and I will do it here locally since I have smaller font and it won't be so overwhelming. And I'm going to move through that hierarchy of namespaces that Java applications have all the way into the 'Config' folder.

Then I'm going to list my files and that is the file I want. I'm going to go ahead and open it up with nano to get some syntax highlighting. And as you can see there, this is a pretty simple file. It simply references, imports some classes as you can see there. And then once we get to the code, that first block that has those dashes is simply figuring out the directories, and this second part is really instantiating that the Debezium configuration. And you can see here if you read through it, and I will leave that up to you, that we are connecting to mySQL Server, that we're specifying the port, that we're setting the password, that we're specifying the database that we're going to be watching and some more information.

But that's really all that this application is doing. So, it's simply going to be watching for changes on that database and its minimal code on top of a default configuration for a web application created through Spring Boot. So, I'm going to go ahead and exit from here and clear my console. And then I'm going to go ahead and run the application. And the command to run the application is the following. So, I'm going to go ahead and hit 'enter' here. And so as you can see there, the application is up and running.

Note that it has noticed that last insert or the only insert that we made into the database, you can see here John Doe, the email, and so on. And so you can see there that this is now up and running. So, let's go ahead and open up Workbench and we're going to go ahead and insert one more row. So, let's open up a new connection and let's go ahead and add that new entry. As you can see there, I'm inserting into 'customerdb' the database, the table, customer, the values of the ID of two, 'Peter Parker', and then 'peter@mit.edu'. So, let me go ahead and run that command. And as you can see there that was inserted. We get the verification here. And if we look at our server, you can see here that all of that information was captured. You can see here Peter Parker, you can see the email plus some additional information that is captured as part of that event. So,

as you can see there, we have successfully created this application. And you should know that Debezium is a very robust CDC, change data capture, application library that can be used and that has tremendous scalability. It has quite a bit of range. It has connectors for most of the widely used data stores. So, it's a great one to understand depth and to be able to learn.

Video 14: Application Architecture (01:25)

When it comes to the architecture of this application, there was a lot that was going on. We created our own network. Into that network, we created a couple of containers. One of those containers had a database. It was sharing ports within that network, and it was also sharing ports with the host. Now, the other container was implementing the Java platform. It was using the Spring Boot type of configuration to be able to develop if we wanted to, but in our case we simply executed it. So, there's a lot going on here. There were several concepts that were key, primarily that of visibility between the containers.

We were also making use of Debezium, one of the most robust packages that exists on the market, and that was monitoring what was taking place on the database. So, this is a pretty good one to spend some time in going through. Now, make sure that you can implement all of these stages. You have all of the code that you need. You've seen piece by piece how it's built. So, this is a pretty good one to spend some time with, to get some comfort with, and to make sure that you understand architecturally the different pieces and how they come together. So now, it's your turn, go ahead and enjoy.